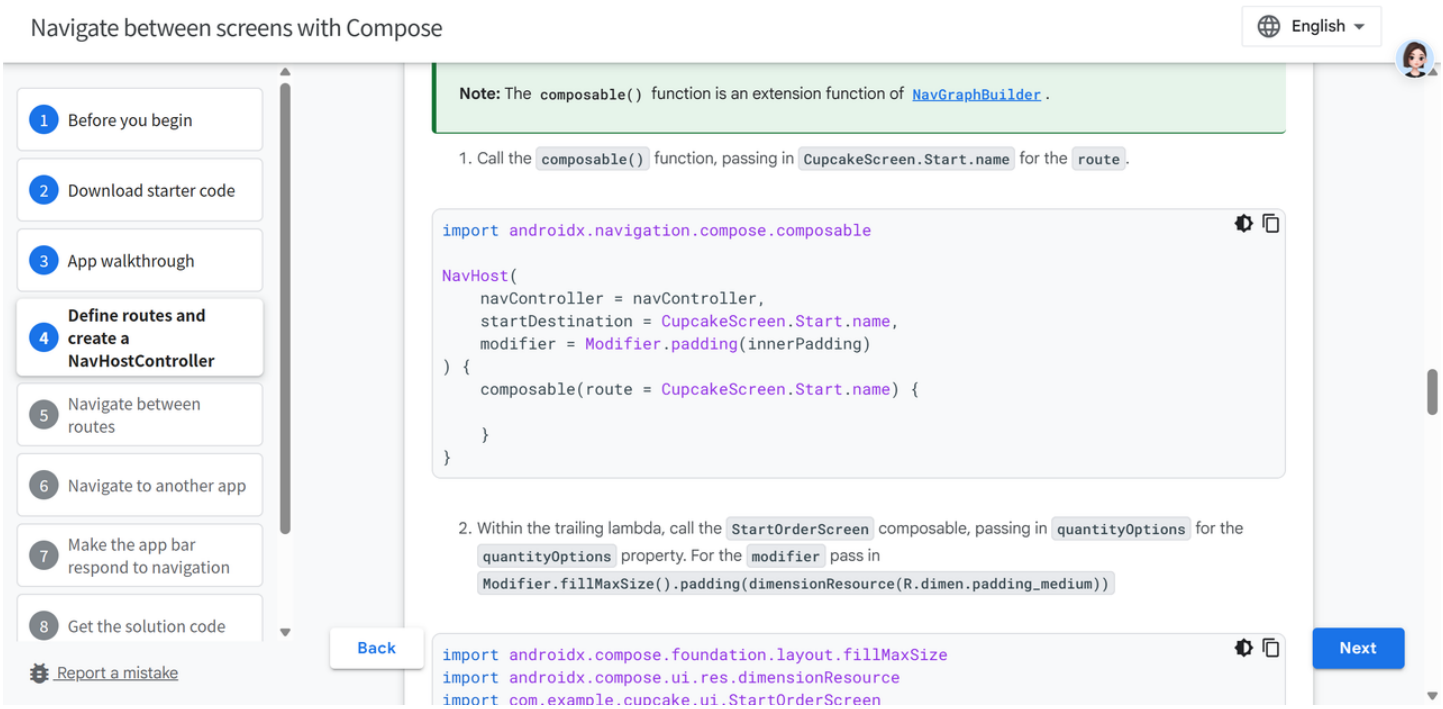
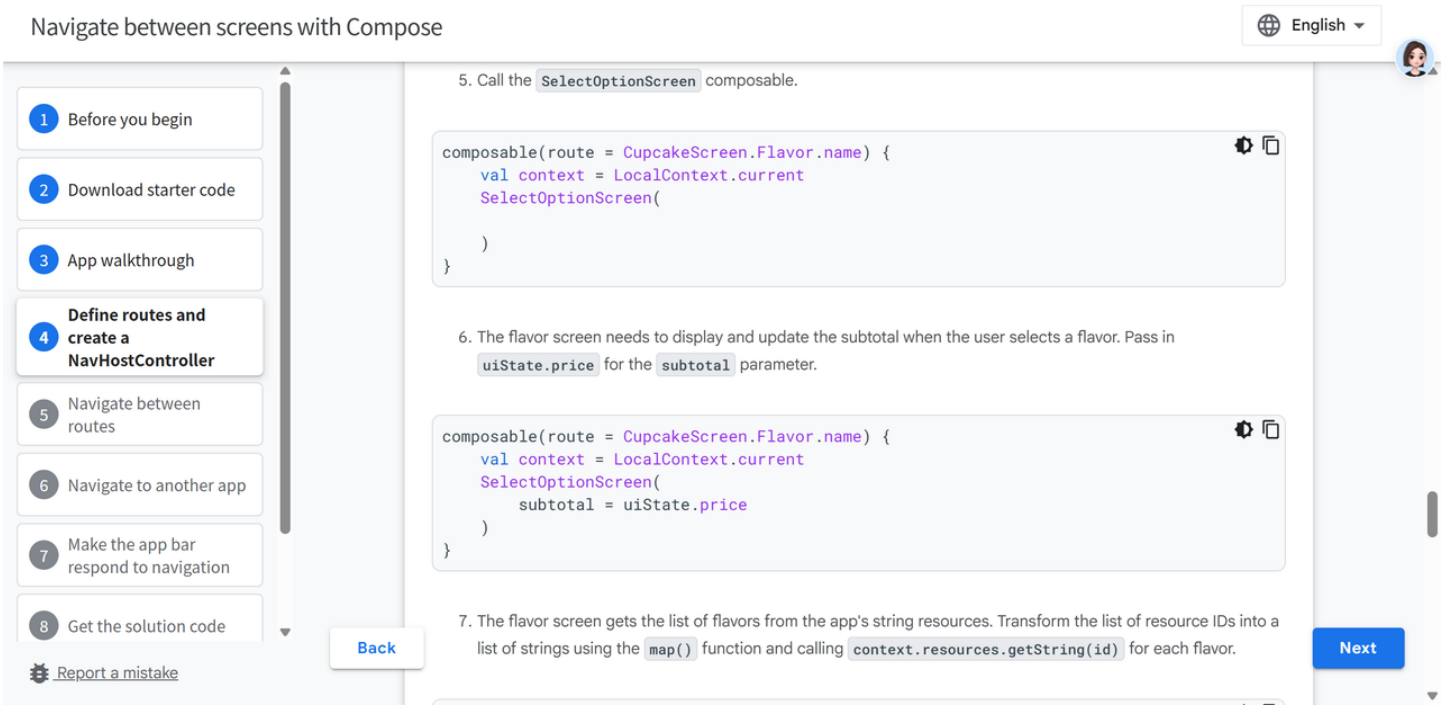
移动开发第九周


Navigate between screens with Compose
English
1 Before you begin
2 Download starter code
3 App walkthrough
4 Define routes and create a NavHostController
5 Navigate between routes
6 Navigate to another app
7 Make the app bar respond to navigation
8 Get the solution code
Report a mistake
Note: The composable() function is an extension function of NavGraphBuilder.
1. Call the composable() function, passing in CupcakeScreen.Start.name for the route.
import androidx.navigation.compose.composable
NavHost(
navController = navController,
startDestination = CupcakeScreen.Start.name,
modifier = Modifier.padding(innerPadding)
) {
composable(route = CupcakeScreen.Start.name) {
}
}
2. Within the trailing lambda, call the StartOrderScreen composable, passing in quantityOptions for the quantityOptions property. For the modifier pass in Modifier.fillMaxSize().padding(dimensionResource(R.dimen.padding_medium))
Back
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.ui.res.dimensionResource
import com.example.cupcake.ui.StartOrderScreen
Next


Navigate between screens with Compose
English
1 Before you begin
2 Download starter code
3 App walkthrough
4 Define routes and create a NavHostController
5 Navigate between routes
6 Navigate to another app
7 Make the app bar respond to navigation
8 Get the solution code
Report a mistake
5. Call the SelectOptionScreen composable.
composable(route = CupcakeScreen.Flavor.name) {
val context = LocalContext.current
SelectOptionScreen(
)
}
6. The flavor screen needs to display and update the subtotal when the user selects a flavor. Pass in uiState.price for the subtotal parameter.
composable(route = CupcakeScreen.Flavor.name) {
val context = LocalContext.current
SelectOptionScreen(
subtotal = uiState.price
)
}
7. The flavor screen gets the list of flavors from the app's string resources. Transform the list of resource IDs into a list of strings using the map() function and calling context.resources.getString(id) for each flavor.
Back
Next

3. Now that the `StartOrderScreen` composable expects a value for `onNextButtonClicked`, find the `StartOrderPreview` and pass an empty lambda body to the `onNextButtonClicked` parameter.

```
@Preview
@Composable
fun StartOrderPreview() {
    CupcakeTheme {
        StartOrderScreen(
            quantityOptions = DataSource.quantityOptions,
            onNextButtonClicked = {},
            modifier = Modifier
                .fillMaxSize()
                .padding(dimensionResource(R.dimen.padding_medium))
        )
    }
}
```

2. Below the `onCancelButtonClicked` parameter, add another parameter of type `() -> Unit` named `onNextButtonClicked` with a default value of `{}`.

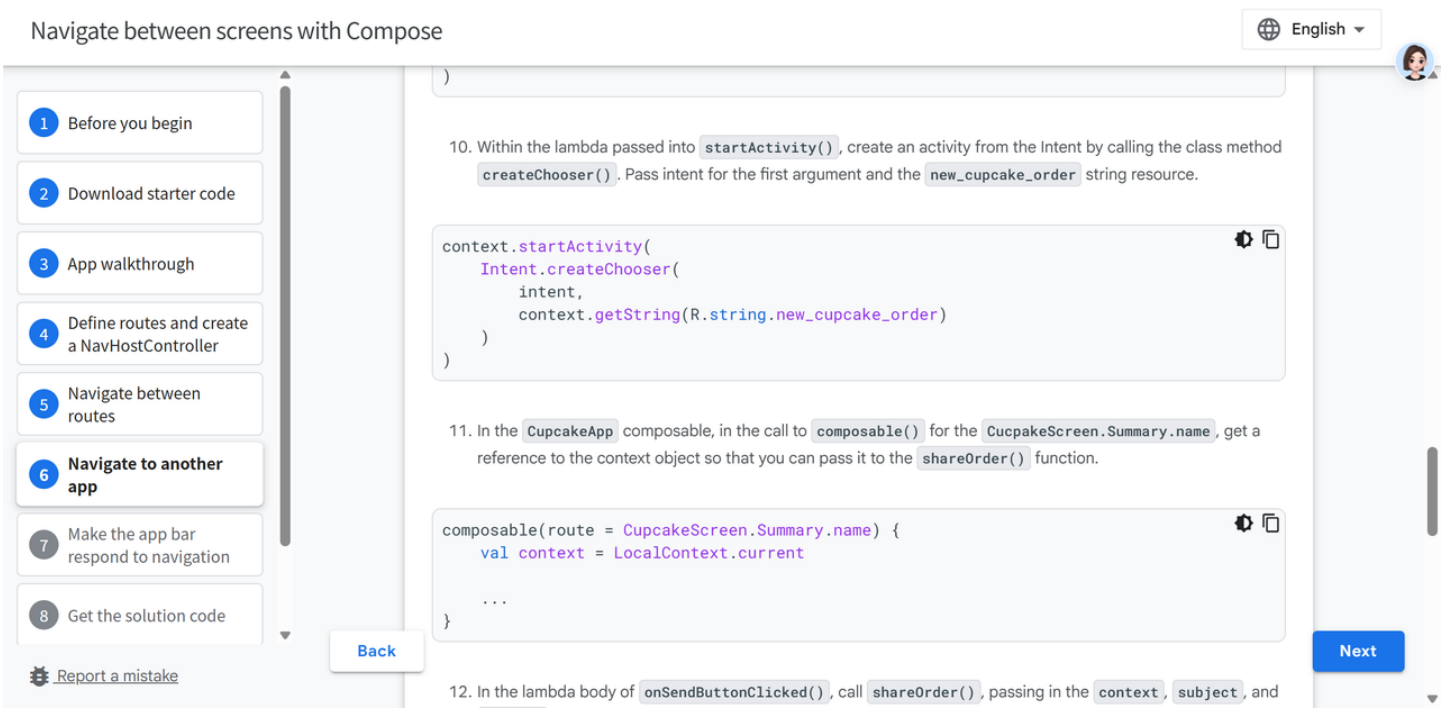
```
@Composable
fun SelectOptionScreen(
    subtotal: String,
    options: List<String>,
    onSelectionChanged: (String) -> Unit = {},
    onCancelButtonClicked: () -> Unit = {},
    onNextButtonClicked: () -> Unit = {},
    modifier: Modifier = Modifier
)
```

3. Pass in `onCancelButtonClicked` for the cancel button's `onClick` parameter.

```
OutlinedButton(
    modifier = Modifier.weight(1f),
    onClick = onCancelButtonClicked
) {
    Text(stringResource(R.string.cancel))
}
```

3. The `OrderSummaryScreen` composable now expects values for `onSendButtonClicked` and `onCancelButtonClicked`. Find the `OrderSummaryPreview`, pass an empty lambda body with two `String` parameters to `onSendButtonClicked` and an empty lambda body to the `onCancelButtonClicked` parameters.

```
@Preview
@Composable
fun OrderSummaryPreview() {
   CupcakeTheme {
       OrderSummaryScreen(
           orderUiState = OrderUiState(0, "Test", "Test", "$300.00"),
           onSendButtonClicked = { subject: String, summary: String -> },
           onCancelButtonClicked = {},
           modifier = Modifier.fillMaxHeight()
       )
   }
}
```



8. Get the solution code

To download the code for the finished codelab, you can use these git commands:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-tra
$ cd basic-android-kotlin-compose-training-cupcake
$ git checkout navigation
```

Alternatively, you can download the repository as a zip file, unzip it, and open it in Android Studio.

Download zip

Note: The solution code is in the `navigation` branch of the downloaded repository.

If you want to see the solution code for this codelab, view it on GitHub.

Back

Next

8. Summary

Congratulations! You've learned how to test the Jetpack Navigation component. You also learned some fundamental skills for writing UI tests, such as writing reusable helper methods, how to leverage `setContent()` to write concise tests, how to set up your tests with the `@Before` annotation, and how to think about maximum test coverage. As you continue to build Android apps, remember to keep writing tests alongside your feature code!

Back

Next step